

# Plano de implementação — Barra HTML com integração Win32

---

## 1. Objetivo

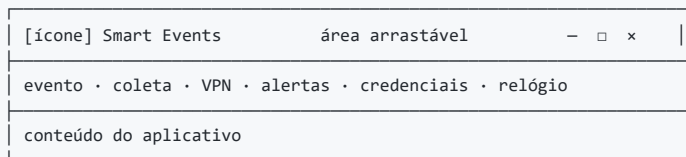
---

Integrar visualmente a barra de título do Windows ao Smart Events, substituindo a barra nativa visível por uma barra desenhada em HTML/CSS e preservando o comportamento esperado de uma janela de produto no Windows.

A implementação continuará usando o stack atual:

- PyWebView 6.2.1 com WebView2;
- backend Python e a API já exposta ao frontend;
- frontend HTML, CSS e JavaScript existente;
- empacotamento PyInstaller em modo `onedir`.

O resultado esperado é uma composição como esta:



A nova barra de janela terá aproximadamente 36 px. O cabeçalho funcional atual continuará existindo logo abaixo. As duas faixas usarão o mesmo sistema visual, mas terão responsabilidades separadas: a primeira controla a janela; a segunda controla o produto.

## 2. Por que o trabalho será dividido em fases

---

A divisão é necessária porque existem três áreas de risco independentes:

1. o hook Win32 precisa provar que funciona corretamente com a janela WinForms criada pelo PyWebView;
2. a barra HTML precisa conversar com o backend sem interferir na API e no cabeçalho existentes;
3. o comportamento precisa continuar correto no executável empacotado, em diferentes versões do Windows, escalas de tela e configurações de monitor.

Cada fase abaixo entrega uma parte verificável e possui uma condição de saída. A barra personalizada só se torna o comportamento padrão depois que todas as fases forem aprovadas.

## 3. Decisões e limites da implementação

---

### 3.1 Decisões já adotadas

- A primeira versão é destinada a Windows 10 e Windows 11.
- A barra de janela será um elemento próprio, separado do `#app-header`.
- A janela continuará sendo reconhecida pelo Windows como redimensionável, minimizável e maximizável.
- O modo de arraste global do PyWebView não será usado; `easy_drag` deverá ficar desabilitado.
- A integração nativa será isolada em um módulo próprio, sem espalhar chamadas `ctypes` pelo restante do projeto.
- O acesso do frontend aos controles da janela passará pelo `bridge.js`.
- O limite atual de `1024 × 600` será preservado inicialmente.
- Sempre existirá um modo de recuperação que usa a barra nativa.

### 3.2 Fora do escopo

- Migrar o shell para Electron, Tauri, Wails ou outro framework.
- Reestruturar o cabeçalho funcional do Smart Events.
- Redesenhar todas as telas do produto.
- Tornar o dashboard utilizável abaixo de 1024 px de largura.
- Criar uma implementação equivalente para macOS ou Linux nesta entrega.

### 3.3 Pré-condição de trabalho

Antes de iniciar cada fase, revisar `git status` e os diffs dos arquivos que serão alterados. O repositório pode conter mudanças em andamento, especialmente em `main.css`, `bridge.js`, `api.py` e arquivos da visão geral. A implementação deve ser aplicada sobre essas mudanças, sem sobrescrevê-las ou descartá-las.

## 4. Arquivos previstos

---

### Novos arquivos

- `core/window_chrome.py`
- `frontend/js/window_chrome.js`
- `frontend/assets/logoSmartEvents-32.png`
- `tools/window_chrome_smoke.py`
- `tests/test_window_chrome.py`
- `tests/test_frontend_window_chrome_ui.py`

## Arquivos que deverão ser alterados

- `main.py`
- `frontend/index.html`
- `frontend/css/main.css`
- `frontend/js/bridge.js`
- `core/self_test.py`
- `README.md`
- possivelmente `main.spec` , somente se o novo asset não for incluído pela regra que já empacota toda a pasta `frontend`

---

## Fase 1 — Prova técnica do frame Win32

---

### Explicação simples

---

Esta fase comprova que é possível retirar a moldura padrão da janela criada pelo PyWebView e devolver os comportamentos nativos do Windows. Ela acontece em uma janela mínima e isolada, sem alterar o Smart Events principal.

Se essa prova não funcionar de maneira estável, a implementação não avança para a interface. Nesse caso, a decisão deve ser reavaliada antes de colocar código experimental no produto.

# Escopo detalhado

---

## Código

1. Criar `tools/window_chrome_smoke.py`.
2. Abrir uma janela PyWebView/WebView2 pequena com HTML embutido e `frameless=True`.
3. Obter o `HWND` por meio de `window.native.Handle` no evento `before_show`.
4. Criar as definições `ctypes` necessárias, usando tipos seguros para processos de 64 bits.
5. Instalar temporariamente um `WNDPROC` intermediário, preservando o procedimento anterior.
6. Fazer o novo procedimento sempre encaminhar mensagens não tratadas para o procedimento original.
7. Experimentar e documentar o tratamento das mensagens:
  - `WM_NCCALCSIZE` ;
  - `WM_NCHITTEST` ;
  - `WM_GETMINMAXINFO` ;
  - `WM_DPICHANGED` ;
  - `WM_SIZE` ;
  - `WM_NCDESTROY` .
8. Preservar ou reativar os estilos:
  - `WS_THICKFRAME` ;
  - `WS_MINIMIZEBOX` ;
  - `WS_MAXIMIZEBOX` ;
  - `WS_SYSMENU` .
9. Aplicar `SWP_FRAMECHANGED` depois da mudança dos estilos.
10. Chamar `DwmDefWindowProc` antes do hit-test personalizado quando isso for necessário.
11. Manter uma referência Python viva para o callback durante toda a vida da janela.
12. Restaurar o `WNDPROC` original antes que a janela seja destruída.
13. Registrar no próprio arquivo as conclusões do experimento, principalmente qualquer diferença encontrada entre Windows 10 e Windows 11.

## Interface

A interface desta fase será propositalmente mínima:

- uma faixa superior de 36 px;
- uma área marcada como arrastável;
- três botões simples;
- uma área central que permita observar o redimensionamento.

Ela não precisa seguir ainda o design definitivo do Smart Events. Seu objetivo é tornar os comportamentos da janela fáceis de observar.

## Testes

1. Extrair o cálculo de hit-test para funções puras e testáveis.
2. Cobrir com testes unitários:
  - borda superior, inferior, esquerda e direita;
  - quatro cantos;
  - área de título;
  - área cliente;
  - região do botão maximizar;
  - coordenadas negativas em monitor posicionado à esquerda do monitor principal;
  - conversão em 96, 120, 144 e 192 DPI.
3. Garantir que um ponto fora da janela nunca seja classificado como controle ou borda.
4. Garantir que o cálculo preserve a prioridade dos cantos sobre as bordas.

## Como validar

---

### Testes automatizados

Executar inicialmente apenas os testes geométricos da prova:

```
.\.venv\Scripts\python.exe -m pytest tests\test_window_chrome.py -q
```

### Validação visual e funcional

Executar:

```
.\.venv\Scripts\python.exe tools\window_chrome_smoke.py
```

Validar manualmente:

- arrastar a janela pela faixa superior;
- arrastar a janela para as bordas do monitor e observar Aero Snap;
- redimensionar pelas quatro bordas e quatro quinas;
- conferir o cursor correto em cada direção;
- dar duplo clique na barra para maximizar e restaurar;
- minimizar e restaurar pela barra de tarefas;
- usar `Alt+Space` ;
- usar `Win+Esquerda` , `Win+Direita` e `Win+Z` ;
- maximizar sem cobrir a barra de tarefas;
- mover entre dois monitores, se disponíveis;
- repetir em 100%, 125%, 150% e 200% de escala.

## Condição para encerrar a fase

A fase é aprovada somente se arraste, redimensionamento, maximização, restauração e encerramento funcionarem sem travamento e sem deixar callbacks ativos depois do fechamento.

Se o Snap Layout não aparecer, a fase deverá confirmar se `WM_NCHITTEST` está retornando `HTMAXBUTTON` na região correta antes de seguir.

## Sugestão de commit

```
test: add Win32 window chrome feasibility harness
```

---

# Fase 2 — Controlador Win32 reutilizável e ciclo de vida seguro

---

## Explicação simples

---

Depois que a prova técnica funcionar, esta fase transforma o experimento em uma unidade de produção. O objetivo é criar um controlador isolado, previsível, testável e capaz de falhar com segurança, sem ainda tornar a barra personalizada o padrão do aplicativo.

# Escopo detalhado

---

## Código

1. Criar `core/window_chrome.py`.
2. Implementar uma classe como `WindowChromeController` com uma API pequena:
  - `attach(window)` ;
  - `detach()` ;
  - `minimize()` ;
  - `toggle_maximize()` ;
  - `close()` ;
  - `get_state()` ;
  - `set_regions(payload)` .
3. Separar a chamada real das funções Win32 em um adaptador interno para permitir mocks nos testes.
4. Tornar `attach()` e `detach()` idempotentes.
5. Impedir instalação em sistemas diferentes de Windows.
6. Impedir instalação quando `window.native` ou o handle ainda não estiver disponível.
7. Preservar os estilos e o `WNDPROC` existentes antes de qualquer alteração.
8. Implementar fallback seguro:
  - registrar o erro completo;
  - não impedir a abertura do aplicativo;
  - voltar à moldura nativa quando o controlador não puder ser instalado.
9. Adicionar resolução explícita do modo da janela:
  - durante desenvolvimento, `--custom-titlebar` ativa a implementação;
  - `--native-titlebar` força a moldura tradicional;
  - se os dois forem fornecidos, o modo nativo vence e o conflito é registrado.
10. Não ativar a barra personalizada por padrão nesta fase.
11. Integrar um diagnóstico específico ao `core/self_test.py` , sem substituir o diagnóstico WebView2 existente.
12. Expor um resultado de diagnóstico legível, por exemplo:

```
{
  "ok": true,
  "mode": "custom",
  "attached": true,
  "dpi": 144,
  "state": "maximized"
}
```

13. Garantir que o encerramento normal continue acionando o mecanismo atual de limpeza e o Job Object usado para subprocessos.

## Interface

Não haverá uma mudança visual definitiva nesta fase. O Smart Events continuará abrindo com a barra nativa no modo padrão.

Quando executado com `--custom-titlebar`, poderá ser usada uma faixa provisória ou o conteúdo mínimo da prova para observar o controlador. Essa opção será tratada como modo técnico até a Fase 3.

## Testes

Criar testes para:

- instalação bem-sucedida;
- dupla chamada de `attach()` ;
- dupla chamada de `detach()` ;
- restauração do `WNDPROC` anterior;
- retenção do callback;
- falha em função Win32 simulada;
- fallback para a moldura nativa;
- execução fora do Windows;
- leitura dos estados normal, maximizado e minimizado;
- alternância maximizar/restaurar;
- validação e limitação dos retângulos recebidos do frontend;
- rejeição de `NaN`, valores negativos inválidos ou regiões maiores que a janela;
- precedência de `--native-titlebar` sobre `--custom-titlebar`.

## Como validar

---

### Testes automatizados

```
.\.venv\Scripts\python.exe -m pytest tests\test_window_chrome.py tests\test_api.py -q
```

Executar também o diagnóstico existente:

```
.\.venv\Scripts\python.exe main.py --self-test
```



## Validação visual e funcional

1. Abrir o Smart Events normalmente e confirmar que nada mudou.
2. Abrir com `--custom-titlebar` e confirmar que o controlador é instalado.
3. Abrir com `--native-titlebar` e confirmar que a moldura tradicional aparece.
4. Forçar uma falha controlada no modo de desenvolvimento e confirmar que o fallback abre a janela nativa.
5. Conferir o log em `data/logs/smart_events.log` e verificar se modo, instalação e eventual fallback foram registrados.

## Condição para encerrar a fase

O controlador deve poder ser ativado e desativado sem travar a janela, e toda falha de inicialização deve terminar em uma janela nativa utilizável.

## Sugestão de commit

```
feat: add safe Win32 window chrome controller
```

---

## Fase 3 — Barra HTML, bridge e controles de janela

---

### Explicação simples

---

Esta fase cria a barra que o operador verá. Ela adiciona o título, o ícone e os controles em HTML/CSS, conecta esses controles ao controlador Win32 e mantém a barra invisível quando o frontend é aberto em um navegador comum.

# Escopo detalhado

---

## Código

1. Adicionar a barra antes de `#app-header` em `frontend/index.html`.
2. Criar `frontend/js/window_chrome.js`.
3. Importar e inicializar o módulo no bootstrap do aplicativo.
4. Adicionar ao `bridge.js` os atalhos:
  - `windowMinimize()` ;
  - `windowToggleMaximize()` ;
  - `windowClose()` ;
  - `windowGetState()` ;
  - `windowSetChromeRegions(payload)` .
5. Adicionar implementações mock inofensivas para desenvolvimento no navegador.
6. Expor as funções de janela com `window.expose(...)` , mantendo-as fora da classe de domínio `Api` sempre que possível.
7. Fazer o módulo enviar ao backend as regiões reais da barra após o carregamento e sempre que houver mudança de tamanho:
  - retângulo total da barra;
  - regiões arrastáveis;
  - botão minimizar;
  - botão maximizar/restaurar;
  - botão fechar.
8. Usar `ResizeObserver` ou um listener de `resize` com debounce para evitar chamadas excessivas.
9. Validar novamente o payload no backend; o frontend não será considerado uma fonte confiável para o hook nativo.
10. Usar a região de maximizar para retornar `HTMAXBUTTON` no Windows 11.
11. Sincronizar o ícone de maximizar/restaurar depois de:
  - clique no botão;
  - duplo clique na barra;
  - Snap;
  - `Win+Setas` ;
  - restauração pela barra de tarefas.
12. Atualizar a versão/cache-buster dos assets alterados para evitar que o perfil persistente do WebView2 carregue CSS ou JavaScript antigos.

## Interface

A barra deverá conter:

- ícone 16 ou 20 px do Smart Events;
- texto "Smart Events";
- região central vazia e arrastável;
- botão minimizar;
- botão maximizar/restaurar;
- botão fechar.

Requisitos visuais:

- altura de 36 px em escala lógica;
- fundo coerente com `--bg-surface` ;
- borda inferior discreta;
- botões com aproximadamente 46 px de largura;
- hover neutro em minimizar e maximizar;
- hover vermelho no botão fechar;
- ícones SVG nítidos em diferentes DPIs;
- estado de foco visível para teclado;
- nenhuma dependência de caracteres de fonte para desenhar os ícones.

Visibilidade:

- a barra aparece quando o shell informa `custom` ;
- fica escondida no navegador comum;
- `?chromePreview=1` permite exibi-la no modo mock para desenvolvimento visual;
- o modo nativo não deve deixar espaço vazio onde a barra personalizada estaria.

O arquivo `assets/logoSmartEvents.ico` deverá ser convertido para um PNG pequeno e colocado em `frontend/assets` , sem alterar a identidade do ícone.

## Testes

Criar `tests/test_frontend_window_chrome_ui.py` usando o servidor estático e Playwright já empregados no projeto.

Cobrir:

- barra escondida no navegador normal;
- barra visível com `?chromePreview=1`;
- presença do título e do ícone;
- `aria-label` dos três controles;
- ordem de tabulação;
- foco visível;
- mudança do ícone maximizar para restaurar;
- chamadas corretas do mock ao clicar nos botões;
- botão fechar não executando uma ação destrutiva no mock;
- envio dos retângulos depois de `resize`;
- retângulos dos controles sem sobreposição;
- região arrastável sem englobar os botões;
- barra correta em 1024, 1440 e 1920 px de largura.

## Como validar

---

### Testes automatizados

```
.\.venv\Scripts\python.exe -m pytest tests\test_window_chrome.py tests\test_frontend_window_chrome_ui.py -q
```

Executar também um smoke frontend mais amplo para detectar regressões no cabeçalho:

```
.\.venv\Scripts\python.exe -m pytest tests\test_frontend_collection_ui.py tests\test_frontend_kpi_overview_ui.py -q
```

### Validação visual

1. Abrir `frontend/index.html?chromePreview=1` pelo servidor de desenvolvimento.
2. Conferir a barra em  $1024 \times 600$ ,  $1440 \times 900$  e  $1920 \times 1080$ .
3. Comparar espaçamento, fundo e altura com o cabeçalho funcional.
4. Verificar hover e foco de cada botão.
5. Confirmar que o botão fechar só fica vermelho durante hover/foco adequado.
6. Abrir o aplicativo com `--custom-titlebar` e repetir os testes da janela da Fase 1.
7. Confirmar que cliques no cabeçalho funcional não arrastam a janela.

### Condição para encerrar a fase

A barra deve estar visualmente pronta, os botões devem controlar a janela real e o frontend comum deve continuar funcionando sem depender do PyWebView.

## Sugestão de commit

feat: add integrated HTML window title bar

# Fase 4 — Integração com layouts, overlays e acessibilidade

## Explicação simples

Adicionar uma nova faixa dentro do viewport muda a altura útil de todas as telas. Esta fase ajusta drawers, modais, visão geral de KPIs e estados responsivos para que nenhum conteúdo fique escondido ou cubra os controles da janela.

## Escopo detalhado

### Código

1. Adicionar tokens no `:root` :

```
--titlebar-h: 36px;  
--hdr-h: 50px;  
--chrome-h: calc(var(--titlebar-h) + var(--hdr-h));
```

2. Fazer `--titlebar-h` valer `0px` quando o modo customizado estiver desligado.
3. Atualizar os drawers que hoje usam apenas `--hdr-h` :
  - alertas;
  - alarmes;
  - logs.
4. Fazer drawers comecem abaixo de `--chrome-h` e descontarem essa altura de `100vh` .
5. Ajustar os modais para preservarem a faixa da janela disponível.
6. Ajustar a visão geral de KPIs, que atualmente usa `100vh` , para ocupar somente a área abaixo da title bar.
7. Manter os controles da janela acima dos overlays do aplicativo.
8. Conferir e ajustar z-indexes para evitar uma escalada de valores desconectados.
9. Impedir que a barra cause scroll vertical no `body` .
10. Garantir que o mapa e os gráficos recebam eventos de resize depois de maximizar, restaurar ou usar Snap.
11. Debouncear os recalculos pesados de Chart.js e Leaflet durante redimensionamento contínuo.
12. Garantir que abrir ou fechar um modal não altere as regiões nativas de hit-test.
13. Adicionar estilos `:focus-visible` e respeitar `prefers-reduced-motion` .
14. Conferir contraste de título, ícones, hover e foco.
15. Manter a largura mínima de 1024 px. Não reduzir o valor nesta fase.

## Interface

Verificar individualmente:

- standby;
- dashboard ativo;
- modo histórico;
- visão geral de KPIs;
- gráfico expandido;
- modal de VPN;
- modais de credenciais;
- seletor de evento;
- confirmação de limpeza;
- drawer de alertas;
- drawer de alarmes;
- drawer de logs;
- toasts.

Comportamento esperado:

- os controles da janela permanecem visíveis mesmo com um modal aberto;
- modais escurecem apenas a área do aplicativo ou seguem uma decisão visual explícita, sem impedir minimizar/fechar;
- drawers nunca ficam por baixo da title bar;
- a visão geral utiliza toda a área disponível abaixo da barra;
- gráficos não ficam cortados depois de maximizar ou restaurar;
- o cabeçalho funcional não perde espaço horizontal para os controles da janela.

## Testes

Adicionar casos Playwright para:

- cálculo da altura útil com e sem title bar;
- top e height dos três drawers;
- visão geral sem overflow vertical inesperado;
- barra acima dos overlays;
- controles da janela clicáveis com modal aberto;
- ausência de sobreposição em 1024 × 600;
- redimensionamento de 1440 × 900 para 1024 × 600;
- abertura e fechamento de cada família de modal;
- foco de teclado sem entrar em regiões ocultas;
- respeito a `prefers-reduced-motion`;
- modo tradicional sem regressão de layout.

Rodar novamente toda a suíte frontend, porque `main.css` é compartilhado por todas as telas.

## Como validar

---

### Testes automatizados

```
.\.venv\Scripts\python.exe -m pytest tests\test_frontend_window_chrome_ui.py tests\test_frontend*_ui.py -q
```

Depois, executar a suíte completa:

```
.\.venv\Scripts\python.exe -m pytest -q
```

### Validação visual

Montar uma grade de verificação com:

- resoluções: 1024 × 600, 1280 × 720, 1440 × 900 e 1920 × 1080;
- escalas do Windows: 100%, 125%, 150% e 200%;
- estados: normal, maximizado, meia tela e restauração;
- telas: standby, dashboard, histórico e visão geral;
- overlays: cada drawer e cada modal relevante.

Em cada combinação, conferir:

- ausência de conteúdo cortado;
- ausência de scroll no documento;
- alinhamento das duas faixas superiores;
- legibilidade do título;
- acesso aos controles da janela;
- atualização correta de mapa e gráficos.

## Condição para encerrar a fase

Todas as telas e overlays devem se adaptar à nova altura sem regressão visual ou funcional, tanto no modo personalizado quanto no modo nativo.

## Sugestão de commit

```
feat: adapt application layout to custom window chrome
```

---

# Fase 5 — Hardening, empacotamento e ativação gradual

---

## Explicação simples

Esta fase transforma a funcionalidade aprovada em comportamento de produção. Ela valida o executável real, adiciona diagnóstico e fallback, documenta o suporte e só então torna a barra personalizada o padrão no Windows.

---

## Escopo detalhado

### Código

1. Estender o diagnóstico para validar o chrome em uma janela WebView2 real.
2. Adicionar um modo específico, se necessário:



3. Fazer o diagnóstico criar, anexar, consultar e desmontar o controlador.
4. Confirmar que o novo asset está presente no bundle PyInstaller.
5. Alterar `main.spec` apenas se a regra `( 'frontend', 'frontend' )` não incluir o asset no artefato final.
6. Registrar no startup:
  - modo solicitado;
  - modo efetivamente usado;
  - versão do Windows;
  - DPI inicial;
  - resultado do attach;
  - motivo de eventual fallback.
7. Não registrar informações sensíveis nem detalhes de credenciais.
8. Atualizar `README.md` com:
  - comportamento da barra;
  - flags de recuperação;
  - limitações conhecidas;
  - procedimento de diagnóstico.
9. Tornar o chrome personalizado padrão somente para Windows/WebView2 após aprovação da matriz manual.
10. Manter `--native-titlebar` como rollback operacional.
11. Manter o comportamento nativo como padrão fora do Windows.
12. Fixar e documentar a compatibilidade com a versão de PyWebView usada no build lock.

## Interface

Fazer uma revisão final de acabamento:

- espaçamento e alinhamento do ícone;
- título correto no shell e na barra de tarefas;
- estados ativo/inativo da janela;
- hover dos botões;
- foco por teclado;
- comportamento quando a janela perde foco;
- ausência de flash da barra nativa durante o startup;
- ausência de salto de layout enquanto o WebView2 carrega;
- ícone correto no executável, barra e taskbar.

Se o attach falhar, a interface deve abrir com a barra nativa e sem a faixa HTML vazia.

## Testes

1. Rodar toda a suíte automatizada.
2. Rodar o self-test no fonte.
3. Gerar o build `onedir` pelo processo oficial do projeto.
4. Rodar o self-test no executável.
5. Rodar o executável em uma máquina limpa ou VM com:
  - Windows 10;
  - Windows 11;
  - WebView2 Runtime instalado;
  - escala diferente de 100%.
6. Testar encerramento com coleta ativa e verificar que não ficam subprocessos Python, Node ou Chromium órfãos.
7. Testar suspensão, bloqueio da sessão e retorno, se isso fizer parte do ambiente operacional.
8. Testar troca de tema do Windows durante a execução.
9. Testar upgrade sobre um perfil WebView2 já existente para detectar cache antigo.
10. Testar explicitamente `--native-titlebar` no executável final.

## Como validar

---

### Testes automatizados

```
.\.venv\Scripts\python.exe -m pytest -q
.\.venv\Scripts\python.exe main.py --self-test
```

Depois do build:

```
.\dist\SmartEvents\SmartEvents.exe --self-test
.\dist\SmartEvents\SmartEvents.exe --self-test-window-chrome
```

Os caminhos exatos do artefato deverão seguir o nome realmente produzido pelo `build.py` / `main.spec`.

### Validação visual do executável

Usar a seguinte checklist:

- ☐ [] abre maximizado sem flash da barra antiga;
- ☐ [] restaura para 1440 × 900;
- ☐ [] respeita o mínimo de 1024 × 600;
- ☐ [] redimensiona pelas oito direções;
- ☐ [] arrasta normalmente;
- ☐ [] Aero Snap funciona;
- ☐ [] `Win+Z` mostra os layouts compatíveis;
- ☐ [] o botão maximizar mostra Snap Layouts no Windows 11;
- ☐ [] minimizar e restaurar pela taskbar funciona;
- ☐ [] `Alt+Space` abre o menu do sistema;
- ☐ [] maximizar não cobre a barra de tarefas;
- ☐ [] trocar de monitor não desloca ou redimensiona incorretamente a janela;
- ☐ [] a barra permanece nítida em 125%, 150% e 200%;
- ☐ [] modais e drawers não cobrem os controles;
- ☐ [] mapa e gráficos se ajustam após resize;
- ☐ [] fechar encerra também os subprocessos;
- ☐ [] `--native-titlebar` abre a versão de recuperação.

## Condição para encerrar a fase

A barra personalizada pode se tornar padrão somente depois que:

- toda a suíte automatizada estiver aprovada;
- o executável passar no self-test;
- Windows 10 e Windows 11 forem validados;
- pelo menos uma configuração com DPI fracionário for validada;
- o fallback pela barra nativa estiver comprovado;
- não houver regressão de encerramento ou processos órfãos.

## Sugestão de commit

```
feat: enable hardened custom window chrome on Windows
```

---

## 5. Matriz consolidada de validação

| Área         | Casos mínimos                                                                                 |
|--------------|-----------------------------------------------------------------------------------------------|
| Sistema      | Windows 10 e Windows 11                                                                       |
| Renderer     | EdgeChromium/WebView2                                                                         |
| DPI          | 100%, 125%, 150% e 200%                                                                       |
| Monitores    | único, dois monitores iguais e dois monitores com DPI diferente                               |
| Estado       | normal, maximizado, minimizado, Snap e restaurado                                             |
| Tamanho      | 1024 × 600, 1280 × 720, 1440 × 900 e 1920 × 1080                                              |
| Entrada      | mouse, teclado, taskbar, <code>Alt+Space</code> , <code>Win+Setas</code> e <code>Win+Z</code> |
| Tela do app  | standby, ativo, histórico e visão geral                                                       |
| Overlay      | drawers, modais, gráfico expandido e toasts                                                   |
| Distribuição | fonte, build <code>onedir</code> e máquina limpa/VM                                           |
| Recuperação  | falha simulada e <code>--native-titlebar</code>                                               |

## 6. Riscos e respostas previstas

### Hook incompatível com atualização do PyWebView

**Risco:** uma futura versão pode mudar os estilos ou o ciclo de criação da janela WinForms.

**Resposta:** usar somente `window.native`, isolar todo acesso Win32, manter a versão travada no build lock e cobrir attach/detach no self-test.

### Callback `WNDPROC` coletado pelo Python

**Risco:** perda da referência pode causar crash nativo.

**Resposta:** armazenar a função callback na instância do controlador até `detach()` / `WM_NCDESTROY`.

### Deadlock entre thread Win32 e chamadas JavaScript

**Risco:** executar `evaluate_js` diretamente dentro do `WNDPROC` pode bloquear a UI.

**Resposta:** o `WNDPROC` não deve fazer chamadas síncronas ao JavaScript. Estado visual será sincronizado por eventos seguros, resize com debounce ou fila posterior.

### DPI e coordenadas incorretas

**Risco:** CSS trabalha em pixels lógicos e Win32 em pixels físicos.

**Resposta:** converter usando `GetDpiForWindow`, tratar coordenadas assinadas e testar monitores com escalas diferentes.

## Snap Layout limitado

**Risco:** a largura mínima atual de 1024 px impede alguns layouts estreitos.

**Resposta:** manter o limite por segurança do dashboard e documentar que somente layouts compatíveis serão aplicados. Uma futura fase de responsividade poderá reduzir o mínimo.

## Cache persistente do WebView2

**Risco:** HTML novo pode ser usado com CSS ou JavaScript antigo.

**Resposta:** atualizar os cache-busters dos assets e manter o mecanismo atual que limita o cache em disco.

## Falha específica de uma máquina

**Risco:** política corporativa, versão do Windows ou runtime pode impedir o hook.

**Resposta:** fallback automático, flag `--native-titlebar`, diagnóstico e logging suficiente para suporte.

## 7. Definição final de pronto

---

A implementação completa estará pronta quando:

1. a barra nativa não aparecer no modo personalizado;
2. o Smart Events mantiver todos os comportamentos essenciais de uma janela Windows;
3. Snap, resize, maximização e restauração funcionarem dentro das limitações da largura mínima;
4. o layout inteiro se ajustar à nova faixa;
5. os testes unitários, frontend e self-tests estiverem aprovados;
6. o executável `onedir` tiver sido validado em Windows 10 e Windows 11;
7. a falha do controlador resultar automaticamente em uma janela nativa utilizável;
8. o suporte puder pedir ao usuário que execute `--native-titlebar` sem reinstalar o produto;
9. não houver regressão na coleta, nos subprocessos ou no encerramento do aplicativo.